

Software Engineering 1

Software Metriken

06.05.2025





Source Code Complexity

Halstead-Metrik

Bei der Halstead-Metrik handelt es sich um eine statisches Analyseverfahren, bei der das Porgramm nicht ausgeführt wird.

Benötigt werden folgende Elemente:

- Anzahl der verwendeten unterschiedlichen Operatoren $n1$ (Variablen und Konstanten)
- Anzahl der verwendeten Operanden $n2$ (Schlüsselwörter, logische und Vergleichsoperatoren)
- $n1 + n2$ bilden zusammen die Vokabulargröße n
- Anzahl der verwendeten Operatoren $N1$
- Anzahl der verwendeten Oparanden $N2$
- $N1 + N2$ bilden zusammen die Implementierungslänge N

Daraus kann man folgendes berechnen:

- Halstead-Länge (HL) $n1 * \log_2 n1 + n2 * \log_2 n2$
- Halstead-Volumen (HV) $N * \log_2 n$

Halstead-Metrik

Aus diesen Werten kann man folgende Kennzahlen ableiten:

- Schwierigkeit ein Programm zu schreiben bzw. zu verstehen, z. B. bei einem Code-Review: $D = \frac{n2}{2} * \frac{N2}{n2}$
- Aufwand $E = D * HV$
- Implementierungszeit $T = \frac{E}{18} \text{ Sekunden}$
- Anzahl ausgelieferter Bugs $B = \frac{HV}{3000}$

Nachteil:

Es werden nur einzelne Funktionen betrachtet und es wird ausschließlich die lexikalische bzw. textuelle Komplexität gemessen

McCabe-Metrik bzw. zyklomatische Komplexität

Die zyklomatische Komplexität gibt die Zahl der unabhängigen Pfade in einem Kontrollflussgraphen an.

Diese kann dazu verwendet werden, um die minimale Anzahl an Testfällen zu bestimmen, damit eine Pfadabdeckung erreicht wird.

Berechnung:

b : Binäre Verzweigungen (if-Statements, Schleifen)

$$C = b + 1$$

Nicht binäre Verzweigungen können auf binäre Verzweigungen heruntergebrochen werden mit:

z : Anzahl Verzweigungen

$$b = z - 1$$

McCabe-Metrik bzw. zyklomatische Komplexität

Bewertung der Zahl:

- **$C < 10$** : Die Methode ist gut testbar und wenig komplex. Dieser Bereich sollte angestrebt werden.
- **$10 < C < 20$** : Die Methode ist noch testbar und mit vertretbarem Aufwand erweiterbar. Sofern es sich um sicherheitsrelevante Methoden handelt, sollte bereits hier erwogen werden, ob die Testbarkeit nicht durch Refaktorisierung erhöht werden kann.
- **$20 < C < 50$** : Die Methode ist komplex, daher kaum erweiterbar und schwer zu testen. Wenn möglich sollte der Code umgebaut werden, um die Komplexität zu senken. In jedem Fall sollten sicherheitsrelevante Methoden refactored werden.
- **$C > 50$** : Solche Methoden gelten als untestbar und nicht erweiterbar. Diese Methoden sollten auf jeden Fall einer refactored werden, da die Fehlerwahrscheinlichkeit sehr hoch ist.

McCabe-Metrik bzw. zyklomatische Komplexität

Nachteil:

Die zyklomatische Komplexität ist ein guter Indikator für komplexen Code, aber kann in gewissen Fällen auch falsche Schlussfolgerungen zulassen.

```
const String wochentagsname(const int nummer)
{
    switch (nummer)
    {
        case 1: return "Montag";
        case 2: return "Dienstag";
        case 3: return "Mittwoch";
        case 4: return "Donnerstag";
        case 5: return "Freitag";
        case 6: return "Samstag";
        case 7: return "Sonntag";
    }
    return "(unbekannter Wochentag)";
}
```

Zyklomatische Komplexität von 8

```
const String wochentagsname(const int nummer)
{
    string[] tage = new string[]
    {
        "Montag",
        "Dienstag",
        "Mittwoch",
        "Donnerstag",
        "Freitag",
        "Samstag",
        "Sonntag"
    };

    String ergebnis = "(unbekannter Wochentag)";
    if ((nummer >= 1) && (nummer <= sizeof (tage)))
    {
        ergebnis = tage[nummer - 1];
    }
    return ergebnis;
}
```

Zyklomatische Komplexität von 2

Weighted methods per class

Gibt die Komplexität einer Klasse an.

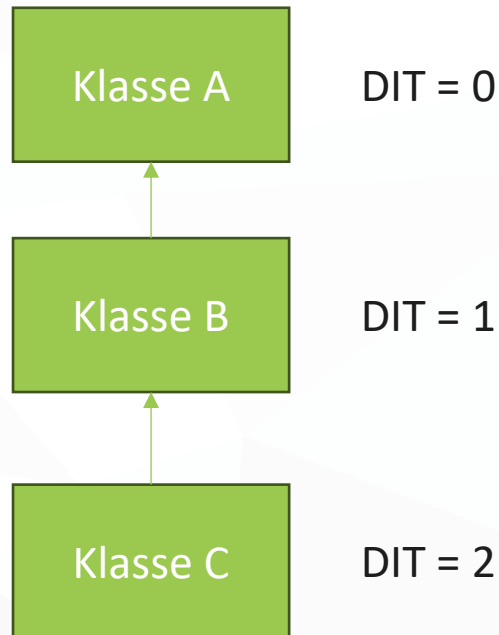
Die Komplexität entspricht der Summe aller zyklomatischen Komplexitäten der Methoden einer Klasse



Relationen zwischen Klassen

Depth of Inheritance Tree (DIT)

Depth of Inheritance Tree (DIT) ist eine Code-Metrik, die spezifisch für die objektorientierte Programmierung ist. Sie misst die maximale Länge zwischen einem Knoten und dem Wurzelknoten in einer Klassenhierarchie.



Einen niedrigen Wert für DIT deutet auf eine niedrige Komplexität hin, aber auch auf weniger Wiederverwendung

Einen hohen Wert für DIT deutet auf ein hohes Maß an Wiederverwendung hin, aber weist auch auf eine hohe Komplexität hin und dadurch ein höheres Fehlerpotenzial.

Es gibt keine fundierte Studie zu einem optimalen Wert für DIT, aber viele Quellen sehen eine obere Grenze bei 5 oder 6.

Number of Children (NOC)

Gibt die Anzahl der direkten Subklassen einer Klasse an.

Es gibt in der Literatur keinen optimalen Wert für NOC.

Je höher der Wert ist desto schwieriger können Änderungen an der Klasse sein, ohne das Verhalten der Subklassen zu verändern.

Daraus folgt, je höher der Wert ist desto mehr muss über Tests abgedeckt werden.

Method Inheritance Factor (MIF)

MIF beschreibt das Verhältnis von vererbten Methoden zu zur Verfügung stehenden Methoden einer Klasse.

$$MIF = \frac{\textit{vererbte Methoden}}{\textit{alle verfügbaren Methoden}}$$

Ein guter Wert liegt laut Literatur zwischen 0,2 und 0,8.

Ein zu hoher Wert deutet auf eine überflüssige Vererbung hin oder ein zu großes Interface der Oberklasse.

Ein zu niedriger Wert deutet auf eine nicht richtig angewandte Vererbung hin (z. B. übermäßig viele Overrides)

Response for a Class (RFC)

Anzahl aller Methoden, die auf einem Objekt einer Klasse aufgerufen werden kann und Anzahl der Methoden, die von den Klassenmethoden aufgerufen werden.

Jede Methode wird nur einmal gezählt, egal wie häufig sie aufgerufen wird.

Ein hoher Wert deutet auf eine sehr komplexe Klasse hin, welche eventuell vereinfacht oder aufgeteilt werden kann. Ebenso impliziert ein hoher Wert einen erhöhten Testaufwand.



Kohäsion

Kohäsion

In der objektorientierten Programmierung beschreibt **Kohäsion**, wie gut eine Programmeinheit eine logische Aufgabe oder Einheit abbildet. In einem System mit starker Kohäsion nimmt jede Programmeinheit (eine Methode, eine Klasse oder ein Modul) genau eine wohldefinierte Aufgabe oder Einheit wahr (siehe Single Responsibility Principle).

Schlechte oder schwache Kohäsion sollte vermieden werden.

Sie führt unter anderem dazu, dass gemeinsame Funktionalitäten einer Klasse nicht wiederverwendet werden, sondern mehrfach umgesetzt werden. Code-Duplizierung ist somit ein Zeichen schwacher Kohäsion. (Vermeidung durch DRY)

Arten von Kohäsion

- **Zufällige Kohäsion** liegt dann vor, wenn die Teile eines Moduls logisch und sachlich unzusammenhängend und so nur zufällig zusammen gruppiert sind (Beispiel: Utility-Klassen).
- **Logische Kohäsion** liegt dann vor, wenn die – an sich unterschiedlichen – Teile eines Moduls logisch durch einen Oberbegriff zusammengefasst werden können (Beispiel: Eingaberoutinen für Maus, Tastatur etc.).
- **Zeitliche Kohäsion** liegt dann vor, wenn die Teile eines Moduls nach ihrer gemeinsamen Ausführungszeit gruppiert sind (Beispiel: Initialisierung/Destruktion innerhalb eines Konstruktors/Destruktors).
- **Prozedurale Kohäsion** liegt dann vor, wenn die Teile eines Moduls nach ihrer Ausführungsreihenfolge gruppiert sind (Beispiel: Eine Funktion, die zunächst Zugriffsrechte prüft und anschließend eine Datei öffnet).
- **Kommunikative Kohäsion** liegt dann vor, wenn die Teile eines Moduls auf gemeinsamen Daten operieren.
- **Sequentielle Kohäsion** liegt dann vor, wenn die Ausgabe eines Teiles eines Moduls die Eingabe eines anderen Modulteiles ist.
- **Funktionale Kohäsion** liegt dann vor, wenn die Teile eines Moduls alle zur Lösung einer einzelnen, wohldefinierten Aufgabe beisteuern.
- **Informationale Kohäsion** ist eine Verstärkung des Begriffs der funktionalen Kohäsion. Sie liegt dann vor, wenn zusätzlich zur funktionalen Kohäsion auch kommunikative Kohäsion für ein Modul vorliegt.

Lack of Cohesion of Methods

Um Kohäsion zu messen, gibt es verschiedene Methoden. LCOM4 ist das am häufigsten verwendete Verfahren, weil es effizienter ist als die LCOM 1-3.

LCOM4 misst die Anzahl an „verbundenen Komponenten“ innerhalb einer Klasse.

Eine verbundene Komponente ist eine Gruppe von miteinander verbundenen Methoden und Klassenvariablen.

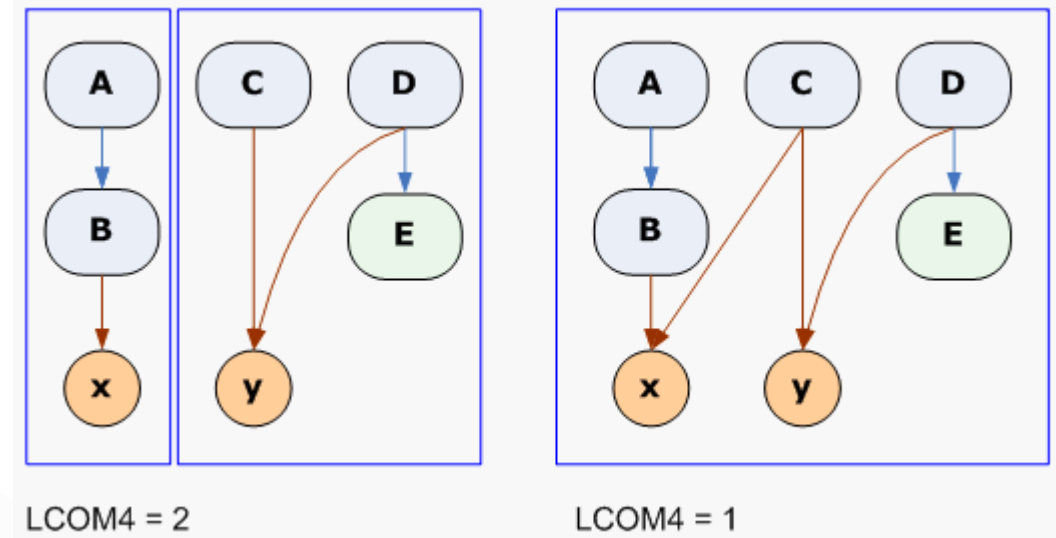
Methoden sind verbunden wenn:

- Wenn beide auf dieselbe Klassenvariable zugreifen
- Wenn Methode A die Methode B aufruft – oder andersherum

LCOM4 = 1: zeigt das es eine kohäsive Klasse ist.

LCOM4 > 2: Die Klasse sollte aufgeteilt werden

LCOM4 = 0: Kommt nur vor wenn, die Klasse keine Methoden hat.





Kopplung

Kopplung von Modulen

Die Kopplung beschreibt, wie stark Module zusammenhängen.

Es lassen sich verschiedene Arten der Kopplung unterscheiden, die sich nach ihrer Stärke ordnen lassen. Dabei gilt die Regel: Je niedriger die Kopplung, desto eigenständiger ist das Modul.

Arten der Kopplung nach Myers absteigend nach Stärke sortiert:

1. **Inhaltskopplung:** Das Modul verlässt sich auf die konkrete inhaltliche Umsetzung eines anderen Moduls bzw. modifiziert die internen Daten des anderen. Diese sehr starke Art der Kopplung sollte unbedingt vermieden werden
2. **Bereichskopplung:** Zwei Module kommunizieren über gemeinsame globale Daten.
3. **Externdatenkopplung:** Zwei Module kommunizieren über einen extern vorgegebenen Mechanismus. Beispielsweise eine bestimmte Datei mit definiertem Aufbau.
4. **Kontrollkopplung:** Ein Modul nimmt Einfluss auf den Kontrollfluss eines anderen. Beispielsweise durch einen Parameter, der die Art der konkreten Aktion spezifiziert.
5. **Datenstrukturkopplung:** Zwei Module kommunizieren über eine komplexe Datenstruktur, von der aber nur ein Teil verwendet wird.
6. **Datenkopplung:** Zwei Module kommunizieren über elementare Daten, beispielsweise einfache Parameter.

Coupling between Objects (CBO)

Bei Coupling between Objects (CBO) wird die Anzahl der Klassen gezählt, die mit einer bestimmten Klasse gekoppelt sind, d. h. bei denen die Methoden der einen Klasse die Methoden der anderen aufrufen oder auf die Variablen der anderen Klasse zugreifen. Da es sich um eine Menge handelt, wird jede Klasse nur einmal gezählt, auch wenn der Verweis in beide Richtungen erfolgt, d. h. wenn A auf B und B auf A verweist, wird B nur einmal gezählt.

Die einzige Überlegung bei der Berechnung dieser Kennzahl ist, ob nur die für das Projekt geschriebenen Klassen oder auch die Bibliotheksklassen berücksichtigt werden sollen.

Generell sollte man eine lose Kopplung und eine hohe Kohäsion anstreben (*loose coupling, high cohesion*)



Test Metriken

Fundamentale Test Metriken

Absolute Zahlen:

- Anzahl der Testfälle
- Anzahl von bestandenen Testfällen
- Anzahl von fehlgeschlagenen Testfällen
- Anzahl blockierter Testfälle
- Anzahl gefundener Defekte/Fehler
- Anzahl akzeptierter Defekte/Fehler
- Anzahl zurückgewiesener Defekte/Fehler
- Anzahl aufgeschobener Defekte/Fehler
- Anzahl kritischer Defekte/Fehler
- Anzahl gefundener Fehler nach der Auslieferung/in Produktion

Test Tracking und Effizienz

- Anteil bestandener Testfälle: $\frac{\text{Anzahl bestandener Testfälle}}{\text{Anzahl ausgeführter Testfälle}}$
- Anteil fehlgeschlagener Testfälle: $\frac{\text{Anzahl fehlgeschlagener Testfälle}}{\text{Anzahl ausgeführter Testfälle}}$
- Anteil blockierter Testfälle: $\frac{\text{Anzahl blockierter Testfälle}}{\text{Anzahl ausgeführter Testfälle}}$
- Anteil behobener Defekte/Fehler: $\frac{\text{Anzahl behobener Defekte}}{\text{Anzahl gemeldeter Defekte}}$
- Anteil akzeptierter Defekte/Fehler: $\frac{\text{Anzahl akzeptierter Defekte}}{\text{Anzahl gemeldeter Defekte}}$
- Anteil zurückgewiesener Defekte/Fehler: $\frac{\text{Anzahl zurückgewiesener Defekte}}{\text{Anzahl gemeldeter Defekte}}$
- Anteil aufgeschobener Defekte/Fehler: $\frac{\text{Anzahl aufgeschobener Defekte}}{\text{Anzahl gemeldeter Defekte}}$
- Anteil kritischer Defekte/Fehler: $\frac{\text{Anzahl kritischer Defekte}}{\text{Anzahl gemeldeter Defekte}}$
- Durchschnittliche Zeit zu beheben von Fehlern: $\frac{\text{Zeit zum Beheben von Fehler}}{\text{Anzahl behobener Fehler}}$
- Effektivität: $\frac{\text{Anzahl gefundener Defekte durch Tests}}{\text{Anzahl gefundener Defekte durch Tests} + \text{Anzahl gefundener Defekte nach der Auslieferung}}$



Verfügbarkeit

Verfügbarkeit

- Durchschnittliche Betriebsdauer zwischen Ausfällen (Mean Time Between Failures, MTBF)
- Durchschnittliche Dauer bis zur Wiederherstellung / Reparatur (Mean time to recovery/repair, MTTR)
- Verfügbarkeit: $\frac{MTBF}{MTBF+MTTR}$

Bei Web-Apps und -Services

- Durchschnittliche Antwort-/Ladezeit